

1. Submission Links

GitHub Repository: <https://github.com/NbscDandoy/Todotask>

Live Link: <https://todotask-1qep.onrender.com/>

2. Entity Relationship Diagram (ERD)

The data model represents a single entity management system for Todo tasks, stored in a relational SQLite database.

[Entity: Todo]

id (PK, Integer) | title (String) | complete (Boolean)

Field	Type	Constraint	Description
id	Integer	Primary Key	Unique identifier for each task.
title	String(200)	Not Null	The text description of the task.
complete	Boolean	Default: False	Completion status of the task.

3. System Flowchart

Mapping the logic from the user interface through the Flask backend to file operations and back.

Step 1: User clicks Delete icon (Route: /delete/id)

Step 2: App fetches Todo title and stores it in session['last_deleted_title']

Step 3: App deletes record from todos.db file and commits.

Step 4: Redirects to Index with ?undone=True parameter.

Step 5: UI displays Undo Toast. If clicked, /undo route re-adds the title to DB.

4. Implementation Algorithm

The system utilizes a **Buffered Persistence Strategy** to allow for single-level undo actions without complex database logic.

1.
Capture Phase: Retrieve task data from DB before deletion.
2.
Buffering Phase: Push string data to the client-side session cookie.
3.
Pruning Phase: Remove row from the SQLite file.
4.
Feedback Phase: Display UI notification based on URL query parameters.
5.
Restoration Phase: Re-insert data from session buffer back into persistent storage.

5. Tech Stack & Environment

- **Backend:** Python / Flask Framework
- **Persistence:** SQLite & SQLAlchemy
- **Frontend:** HTML5, CSS3, Bootstrap Icons
- **Hosting:** Render.com (Gunicorn)

6. Deployment Instructions

```
# Clone project  
git clone https://github.com/NbscDandoy/Todotask.git
```

```
# Install packages  
pip install flask flask-sqlalchemy
```

```
# Launch Application  
python app.p
```